

Title: EDA Over Bike Details Dataset



```
In [42]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings("ignore")
```

```
In [44]: df = pd.read_csv("BIKE DETAILS.csv")
```

```
In [46]: df
```

```
Out[46]:
```

	name	selling_price	year	seller_type	owner	km_driven	ex_showrc
0	Royal Enfield Classic 350	175000	2019	Individual	1st owner	350	
1	Honda Dio	45000	2017	Individual	1st owner	5650	
2	Royal Enfield Classic Gunmetal Grey	150000	2018	Individual	1st owner	12000	
3	Yamaha Fazer FI V 2.0 [2016-2018]	65000	2015	Individual	1st owner	23000	
4	Yamaha SZ [2013-2014]	20000	2011	Individual	2nd owner	21000	
...	...	...	...	...	...	...	
1056	Activa 3g	17000	2010	Individual	1st owner	500000	
1057	Honda CB twister	16000	2012	Individual	1st owner	33000	
1058	Bajaj Discover 125	15000	2013	Individual	2nd owner	35000	
1059	Honda CB Shine	12000	2009	Individual	1st owner	53000	
1060	Bajaj Pulsar 150	10000	2008	Individual	1st owner	92233	

1061 rows × 7 columns

```
In [48]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1061 entries, 0 to 1060
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   name                   1061 non-null   object
1   selling_price          1061 non-null   int64
2   year                   1061 non-null   int64
3   seller_type            1061 non-null   object
4   owner                  1061 non-null   object
5   km_driven              1061 non-null   int64
6   ex_showroom_price      626 non-null    float64
dtypes: float64(1), int64(3), object(3)
memory usage: 58.2+ KB

```

```
In [50]: df.isna().sum()
```

```

Out[50]: name                0
selling_price              0
year                      0
seller_type               0
owner                     0
km_driven                 0
ex_showroom_price        435
dtype: int64

```

```
In [52]: df.shape
```

```
Out[52]: (1061, 7)
```

```
In [54]: df.ex_showroom_price.value_counts()
```

```

Out[54]: ex_showroom_price
50267.0      23
85011.0      20
54605.0      15
74295.0      14
78712.0      12
..
129662.0     1
66697.0      1
69983.0      1
76632.0      1
32000.0      1
Name: count, Length: 230, dtype: int64

```

```
In [56]: df.ex_showroom_price
```

```
Out[56]: 0      NaN
        1      NaN
        2    148114.0
        3    89643.0
        4      NaN
        ...
       1056    52000.0
       1057    51000.0
       1058    57000.0
       1059    58000.0
       1060    75000.0
       Name: ex_showroom_price, Length: 1061, dtype: float64
```

```
In [58]: #1 What is the range of selling prices in the dataset?
        range_sp = df.selling_price.max()-df.selling_price.min()
        range_sp
```

```
Out[58]: 755000
```

```
In [60]: #2 What is the median selling price for bikes in the dataset?
        df.selling_price.median()
```

```
Out[60]: 45000.0
```

```
In [62]: #3 What is the most common seller type?
        df.seller_type.value_counts()
```

```
Out[62]: seller_type
        Individual    1055
        Dealer         6
        Name: count, dtype: int64
```

```
In [64]: # the indivisual is the most common seller type
```

```
In [66]: #4 How many bikes have driven more than 50,000 kilometers?
        (df.km_driven > 50000).sum()
```

```
Out[66]: 170
```

```
In [68]: #5 What is the average km_driven value for each ownership type?
        df.groupby('owner').agg({'km_driven': 'mean'})
```

```
Out[68]:
```

	km_driven
owner	
1st owner	32816.583333
2nd owner	39288.991870
3rd owner	33292.181818
4th owner	311500.000000

```
In [70]: #6 What proportion of bikes are from the year 2015 or older?
new = (df.year >= 2015).sum()
old = (df.year < 2015).sum()
total = len(df)
new_proportion = old*100/total
new_proportion
```

Out[70]: 47.219604147031106

```
In [72]: # Question #7: What is the trend of missing values across the dataset?
# The dataset has 435 missing values in the 'ex_showroom_price' column.
# To analyze their trend, we will extract only the rows with missing values
# and study their distribution across other features.
df_m = df[df.ex_showroom_price.isnull()]
df_m
```

Out[72]:

	name	selling_price	year	seller_type	owner	km_driven	ex_showroc
--	------	---------------	------	-------------	-------	-----------	------------

0	Royal Enfield Classic 350	175000	2019	Individual	1st owner	350	
1	Honda Dio	45000	2017	Individual	1st owner	5650	
4	Yamaha SZ [2013-2014]	20000	2011	Individual	2nd owner	21000	
7	Royal Enfield Bullet 350 [2007-2011]	180000	2008	Individual	2nd owner	39000	
8	Hero Honda CBZ extreme	30000	2010	Individual	1st owner	32000	
...	...	...	...	...	...	...	
951	Royal Enfield Electra 5 S	90000	2012	Individual	1st owner	40000	
952	Hero Honda Hunk	20000	2010	Individual	1st owner	17000	
957	Bajaj Pulsar 220 DTS-i	60000	2014	Individual	1st owner	16000	
958	Hero Honda CBZ extreme	40000	2009	Individual	1st owner	50000	
959	Hero Honda CBZ extreme	35000	2012	Individual	1st owner	60000	

435 rows × 7 columns

```

In [74]: missing_per_year = df[df['ex_showroom_price'].isnull()].groupby('year').size()

plt.figure(figsize=(8,5), facecolor="lightgrey") # set background color

# Plot the line
plt.plot(missing_per_year.index, missing_per_year.values, color="skyblue", mar

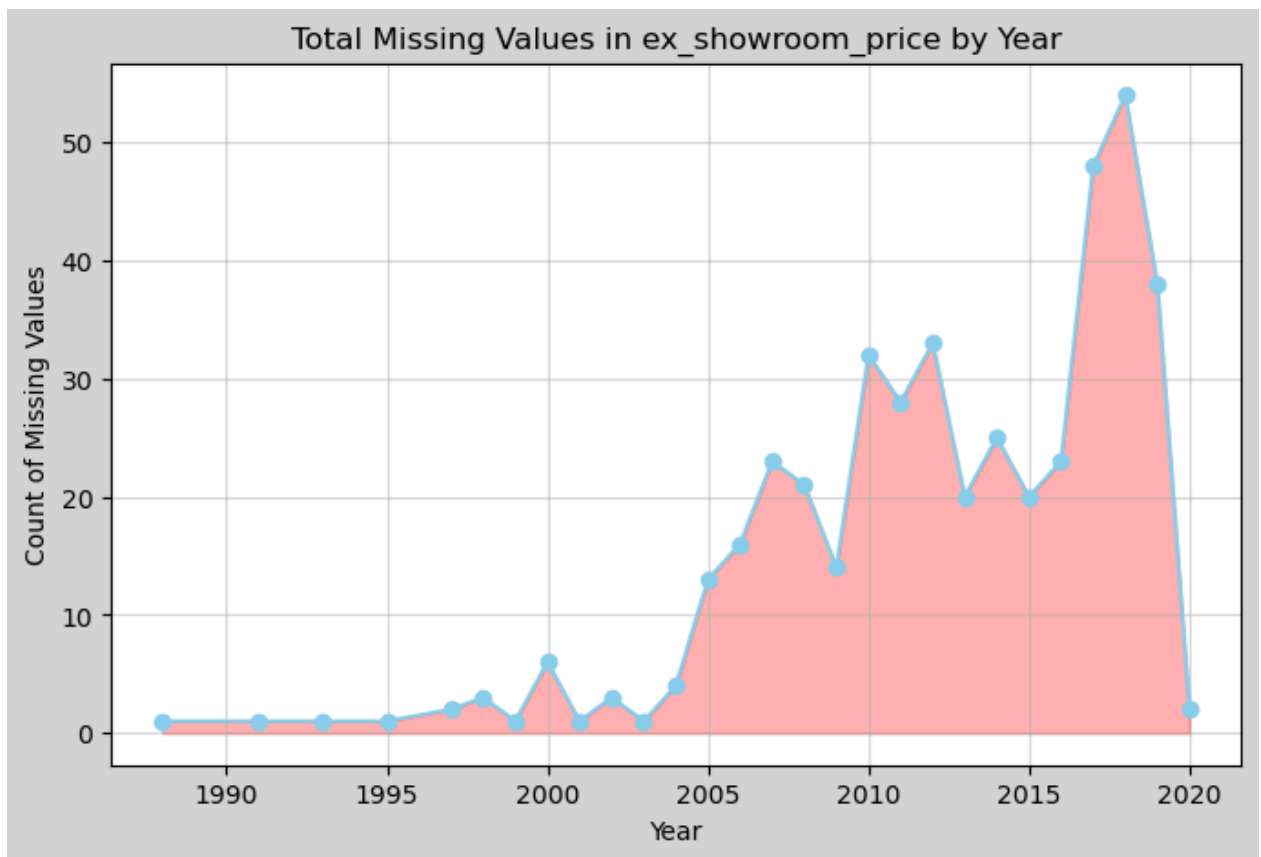
# Fill area under the line
plt.fill_between(missing_per_year.index, missing_per_year.values, color="red",

# Titles and labels
plt.title("Total Missing Values in ex_showroom_price by Year")
plt.xlabel("Year")
plt.ylabel("Count of Missing Values")

# Grid
plt.grid(True, linestyle='--', alpha=0.5)

plt.show()

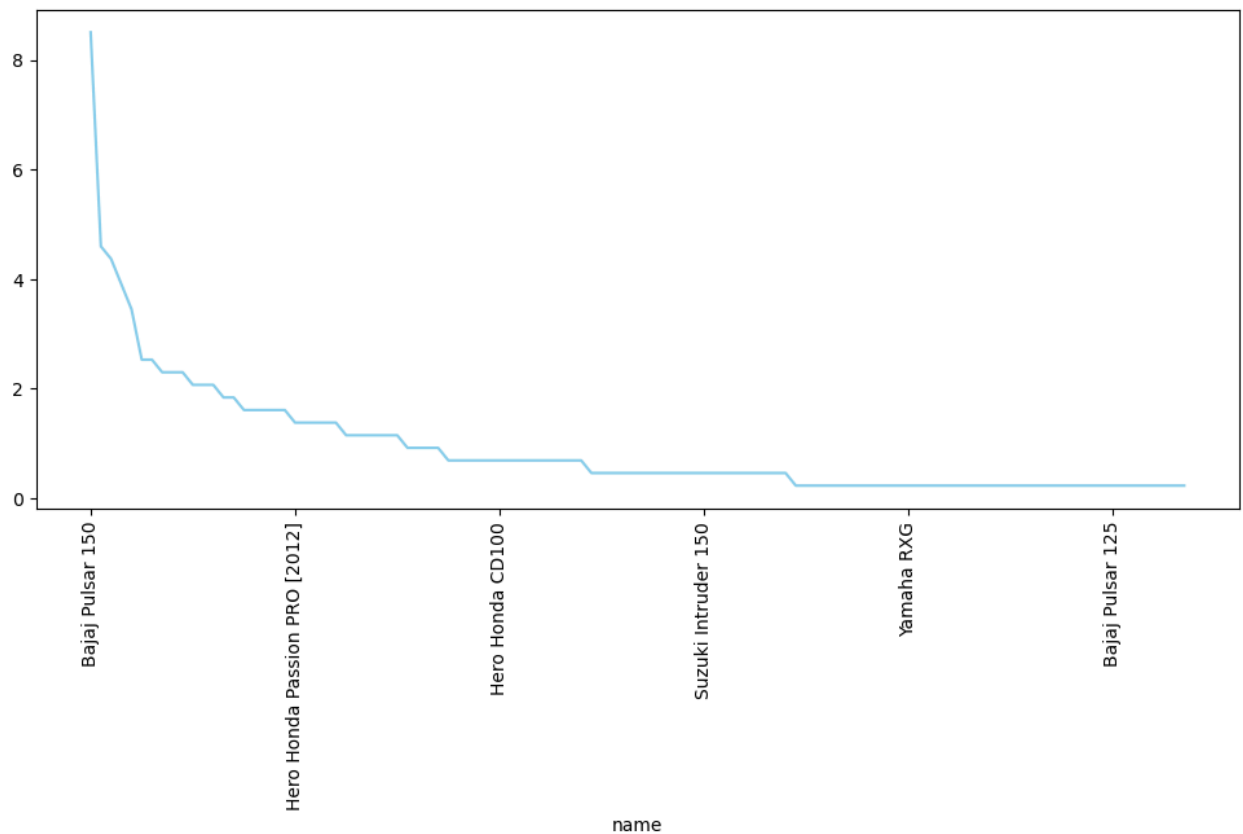
```



```

In [76]: #trend over name column
(df_m['name'].value_counts(normalize=True) * 100).plot(kind='line', figsize=(1
plt.xticks(rotation = 90)
plt.show()

```



```
In [78]: #8 What is the highest ex_showroom_price recorded, and for which bike?
# Find the row with the highest ex_showroom_price
df.loc[df['ex_showroom_price'].idxmax(), ['name', 'ex_showroom_price']]
```

```
Out[78]: name          Harley-Davidson Street Bob
ex_showroom_price      1278000.0
Name: 134, dtype: object
```

```
In [82]: #9 What is the total number of bikes listed by each seller type?
df.seller_type.value_counts()
```

```
Out[82]: seller_type
Individual    1055
Dealer        6
Name: count, dtype: int64
```

```
In [84]: #10 What is the relationship between selling_price and km_driven for first-own
first_owner_df = df.loc[df['owner']=='1st owner',['selling_price', 'km_driven']]
```

```
In [86]: first_owner_df
```

Out[86]:

	selling_price	km_driven
0	175000	350
1	45000	5650
2	150000	12000
3	65000	23000
5	18000	60000
...	...	...
1055	18000	35000
1056	17000	500000
1057	16000	33000
1059	12000	53000
1060	10000	92233

924 rows × 2 columns

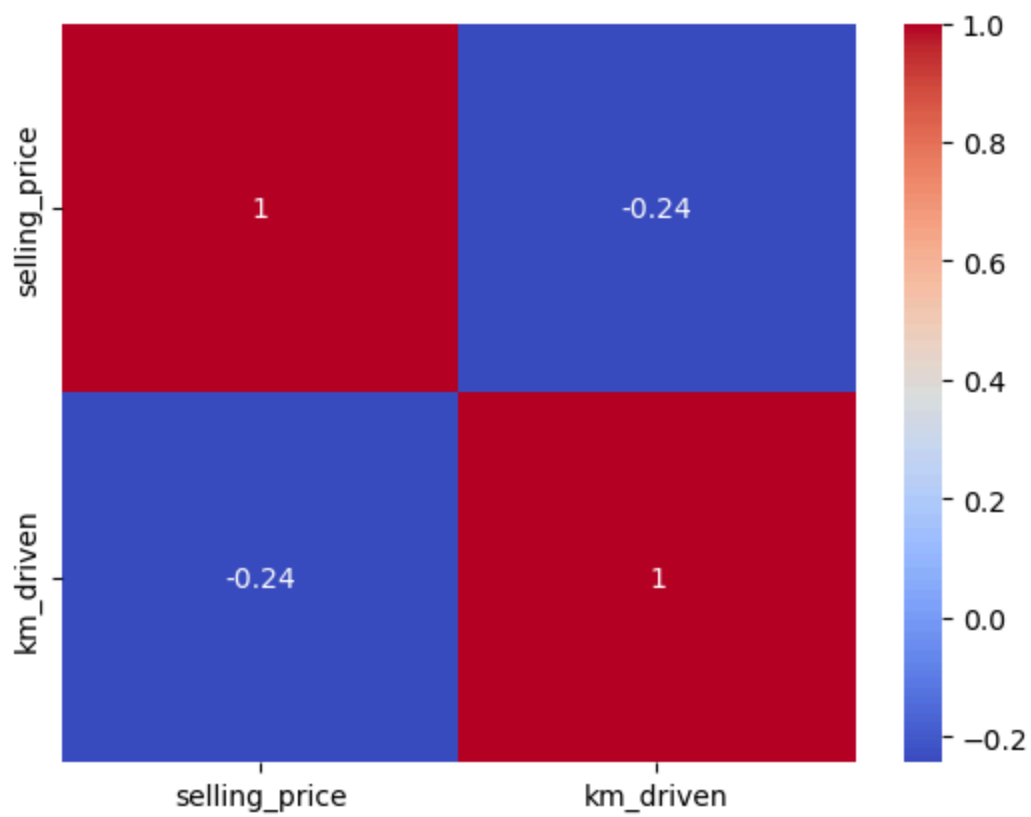
```
In [105... first_owner_df.corr()
```

Out[105...]

	selling_price	km_driven
selling_price	1.000000	-0.243482
km_driven	-0.243482	1.000000

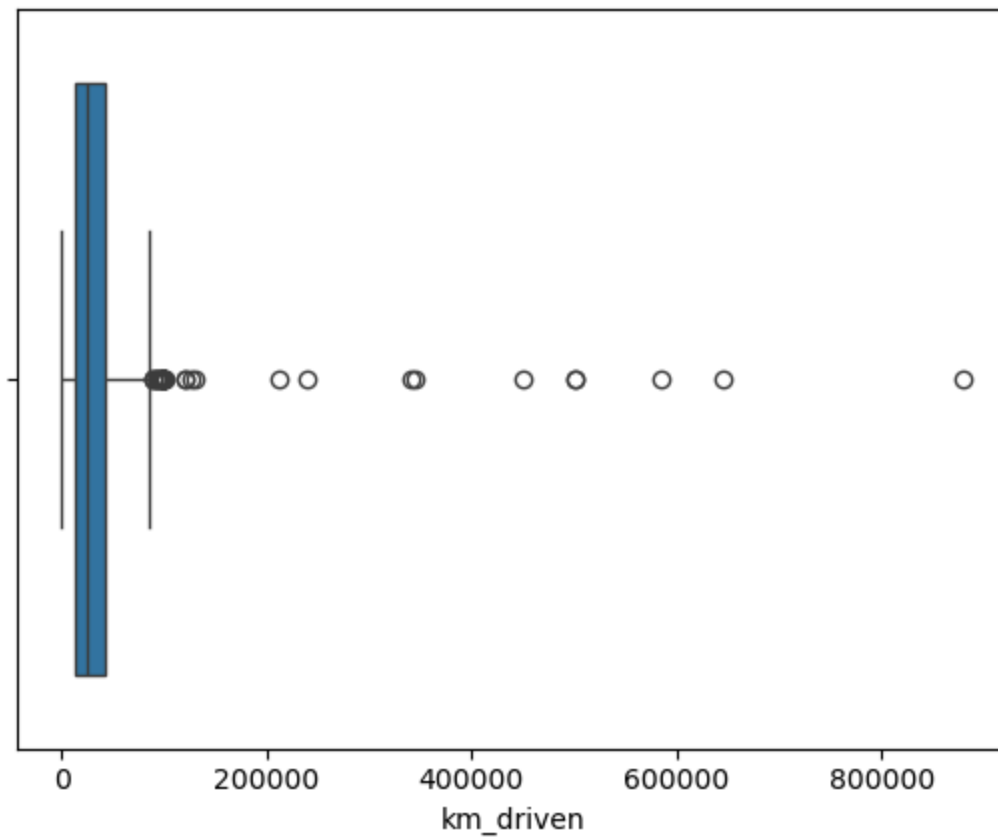
```
In [109... sns.heatmap(first_owner_df.corr(),annot = True, cmap = 'coolwarm')
```

Out[109... <Axes: >



```
In [123... #11 Identify and remove outliers in the km_driven column using the IQR method.  
sns.boxplot(x=df['km_driven'])
```

```
Out[123... <Axes: xlabel='km_driven'>
```

```
In [131... Q1 = df.km_driven.quantile(0.25)
Q3 = df.km_driven.quantile(0.75)
IQR = Q3-Q1
```

```
In [135... lower_bound = Q1-1.5*IQR
upper_bound = Q3+1.5*IQR
```

```
In [155... outlier = df[(df.km_driven<lower_bound) | (df.km_driven>upper_bound)]
outlier.km_driven #these are the required outlier
```

```
Out[155... 91      500000
          93      130000
          115     120000
          135       90000
          154       99000
          184     100000
          363     646000
          470       95000
          474       90000
          477     585659
          511     100000
          535     100000
          548     100000
          575     340000
          592     101000
          598     120000
          610     100000
          630       98000
          642     880000
          668       93000
          687     100000
          695     100000
          712     100000
          714     240000
          716     100000
          726     100000
          773       95000
          782     100000
          797     345000
          830     100000
          872       93000
          877     450000
          901     100000
          903     127000
          908       89000
          930     100000
         1039     213000
         1056     500000
         1060       92233
Name: km_driven, dtype: int64
```

```
In [157... #Remove these outlier
df_no_outliar = df[(df.km_driven >= lower_bound) | (df.km_driven<=upper_bound)]
df_no_outliar
```

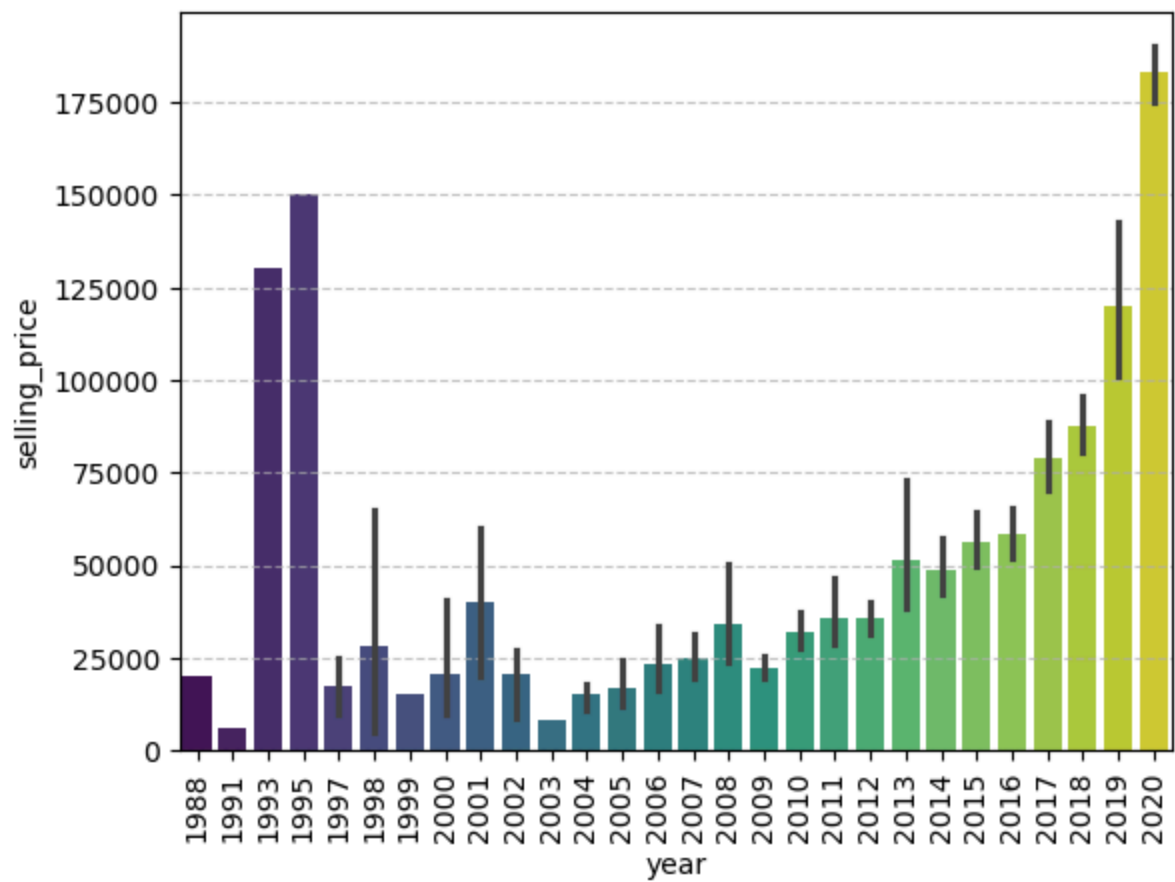
Out[157...

	name	selling_price	year	seller_type	owner	km_driven	ex_showrc
0	Royal Enfield Classic 350	175000	2019	Individual	1st owner	350	
1	Honda Dio	45000	2017	Individual	1st owner	5650	
2	Royal Enfield Classic Gunmetal Grey	150000	2018	Individual	1st owner	12000	
3	Yamaha Fazer FI V 2.0 [2016-2018]	65000	2015	Individual	1st owner	23000	
4	Yamaha SZ [2013-2014]	20000	2011	Individual	2nd owner	21000	
...	...	...	...	...	...	...	
1056	Activa 3g	17000	2010	Individual	1st owner	500000	
1057	Honda CB twister	16000	2012	Individual	1st owner	33000	
1058	Bajaj Discover 125	15000	2013	Individual	2nd owner	35000	
1059	Honda CB Shine	12000	2009	Individual	1st owner	53000	
1060	Bajaj Pulsar 150	10000	2008	Individual	1st owner	92233	

1061 rows × 7 columns

In [182...

```
#12 Perform a bivariate analysis to visualize the relationship between year and selling price
sns.barplot(x='year',y='selling_price',data=df,palette = 'viridis')
plt.xticks(rotation = 90)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



In [192... *#13 What is the average depreciation in selling price based on the bike's age*
`df["bike's age"] = 2025-df['year']`
`df`

Out[192...

	name	selling_price	year	seller_type	owner	km_driven	ex_showrc
0	Royal Enfield Classic 350	175000	2019	Individual	1st owner	350	
1	Honda Dio	45000	2017	Individual	1st owner	5650	
2	Royal Enfield Classic Gunmetal Grey	150000	2018	Individual	1st owner	12000	
3	Yamaha Fazer FI V 2.0 [2016-2018]	65000	2015	Individual	1st owner	23000	
4	Yamaha SZ [2013-2014]	20000	2011	Individual	2nd owner	21000	
...	...	...	...	...	...	...	
1056	Activa 3g	17000	2010	Individual	1st owner	500000	
1057	Honda CB twister	16000	2012	Individual	1st owner	33000	
1058	Bajaj Discover 125	15000	2013	Individual	2nd owner	35000	
1059	Honda CB Shine	12000	2009	Individual	1st owner	53000	
1060	Bajaj Pulsar 150	10000	2008	Individual	1st owner	92233	

1061 rows × 9 columns

In [206...

```
df_age_depre = df.groupby("bike's age")['selling_price'].mean().round(2).reset_index()
```

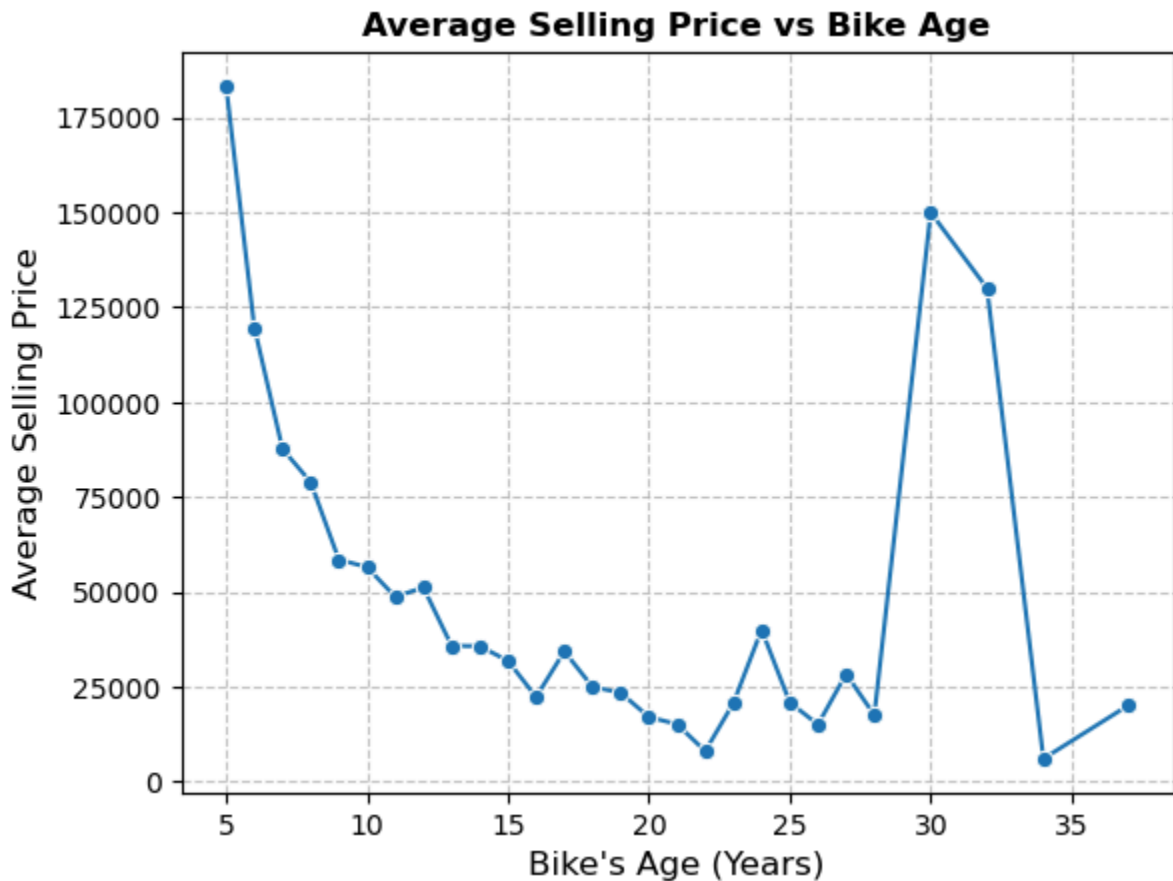
Out[206...

	bike's age	avg_selling_price
0	5	183333.33
1	6	119689.51
2	7	87660.37
3	8	78894.74
4	9	58469.02
5	10	56500.00
6	11	48668.13
7	12	51136.99
8	13	35748.40
9	14	35655.72
10	15	31793.33
11	16	22267.86
12	17	34289.29
13	18	24927.59
14	19	23380.00
15	20	16978.57
16	21	15100.00
17	22	8000.00
18	23	20666.67
19	24	40000.00
20	25	20833.33
21	26	15000.00
22	27	28333.33
23	28	17500.00
24	30	150000.00
25	32	130000.00
26	34	6000.00
27	37	20000.00

In [219...

```
sns.lineplot(x="bike's age", y = 'avg_selling_price',data = df_age_depre, mark
plt.title("Average Selling Price vs Bike Age",fontweight = 'bold')
plt.xlabel("Bike's Age (Years)",fontsize = 12)
plt.ylabel("Average Selling Price",fontsize = 12)
```

```
plt.grid(True,linestyle = '--',alpha = 0.7)
plt.show()
```



```
In [245... #14 Which bike names are priced significantly above the average price for thei
df['avg_sp'] = df.groupby('year')['selling_price'].transform('mean').round(2)
above_avg = df[df['selling_price'] > df['avg_sp']]
above_avg['name']
```

```
Out[245... 0          Royal Enfield Classic 350
2      Royal Enfield Classic Gunmetal Grey
3          Yamaha Fazer FI V 2.0 [2016-2018]
7      Royal Enfield Bullet 350 [2007-2011]
13          Yamaha YZF R3
...
1004          Bajaj Pulsar NS 200
1005          TVS Apache RTR 160
1008          Bajaj Pulsar 220 F
1012          Bajaj Pulsar NS 200
1023          Bajaj Avenger 220 dtsi
Name: name, Length: 334, dtype: object
```

```
In [248... #15 Develop a correlation matrix for numeric columns and visualize it using a
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1061 entries, 0 to 1060
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   name                   1061 non-null   object
1   selling_price          1061 non-null   int64
2   year                   1061 non-null   int64
3   seller_type            1061 non-null   object
4   owner                  1061 non-null   object
5   km_driven              1061 non-null   int64
6   ex_showroom_price      626 non-null    float64
7   age                    1061 non-null   int64
8   bike's age             1061 non-null   int64
9   avg_sp                 1061 non-null   float64
dtypes: float64(2), int64(5), object(3)
memory usage: 83.0+ KB

```

```

In [254... numeric_cols = df.select_dtypes(include='number')
           corr_matrix = numeric_cols.corr()

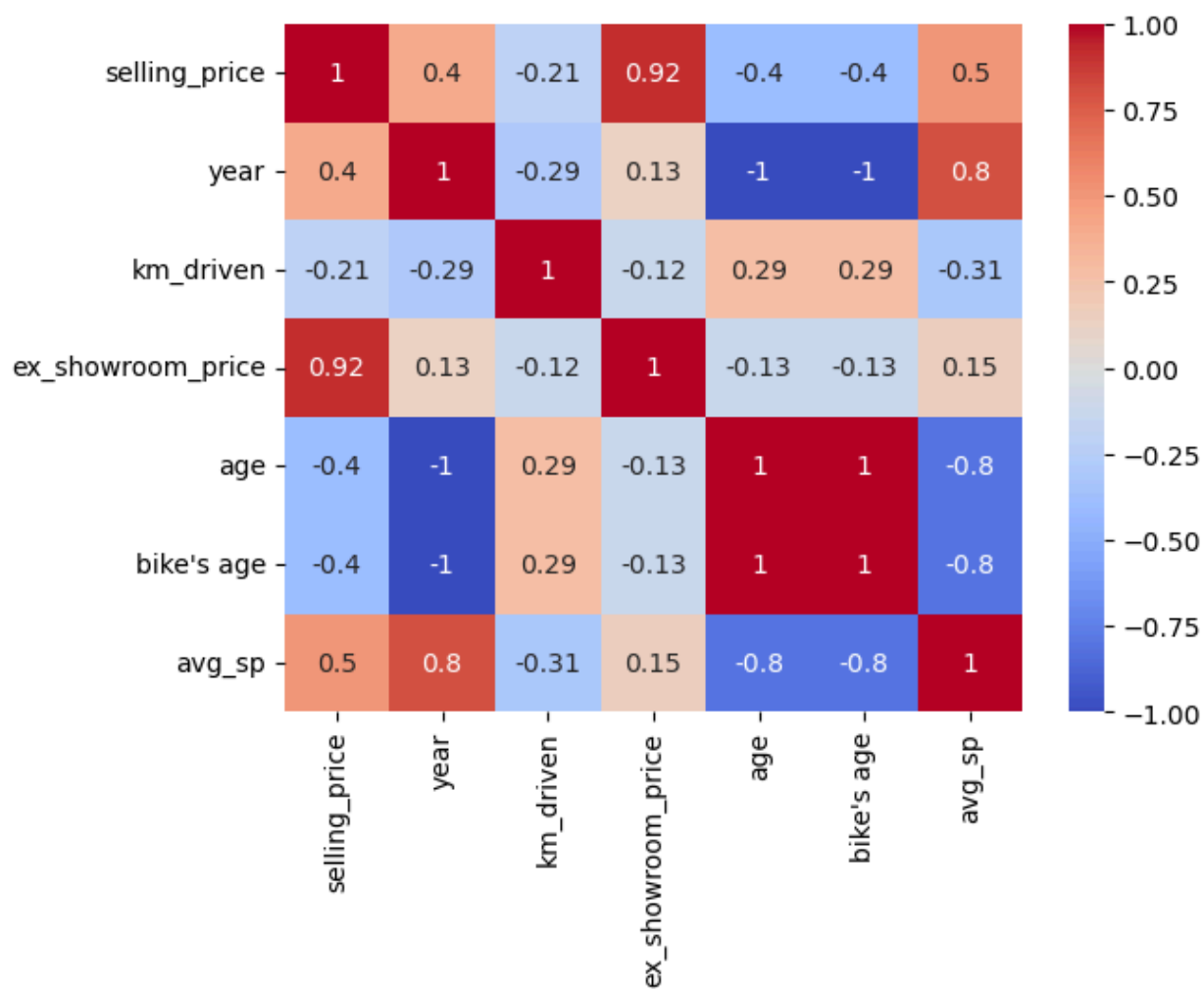
           sns.heatmap(corr_matrix ,annot = True, cmap = 'coolwarm')

```

```

Out[254... <Axes: >

```

In []:

Title: EDA Over Car Sale Dataset



In []:

```
In [148... import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")
```

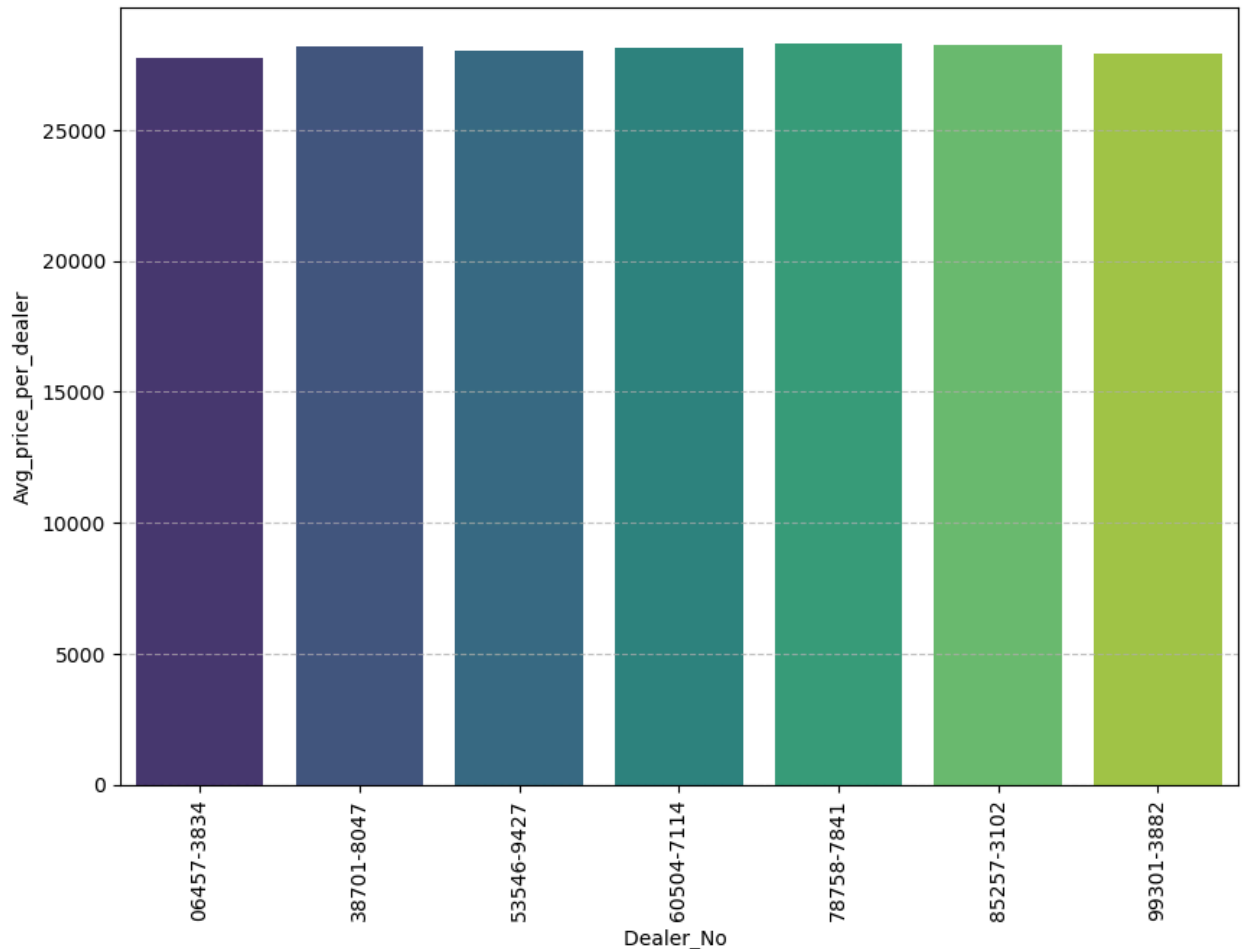
```
In [150... df = pd.read_csv("Car Sale.csv")
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23906 entries, 0 to 23905
Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Car_id                23906 non-null  object
1   Date                  23906 non-null  object
2   Customer Name         23905 non-null  object
3   Gender                23906 non-null  object
4   Annual Income         23906 non-null  int64
5   Dealer_Name           23906 non-null  object
6   Company               23906 non-null  object
7   Model                 23906 non-null  object
8   Engine                23906 non-null  object
9   Transmission          23906 non-null  object
10  Color                 23906 non-null  object
11  Price ($)              23906 non-null  int64
12  Dealer_No              23906 non-null  object
13  Body Style            23906 non-null  object
14  Phone                 23906 non-null  int64
15  Dealer_Region         23906 non-null  object
dtypes: int64(3), object(13)
memory usage: 2.9+ MB
```

```
In [151... #1 What is the average selling price of cars for each dealer, and how does it
df_dealer = df.groupby(["Dealer_No"])[["Price ($)"]].mean().round(2).reset_index
df_dealer
```

```
Out[151...   Dealer_No  Avg_price_per_dealer
0  06457-3834          27771.61
1  38701-8047          28208.62
2  53546-9427          28009.71
3  60504-7114          28134.03
4  78758-7841          28272.41
5  85257-3102          28259.14
6  99301-3882          27920.85
```

```
In [154... plt.figure(figsize=(10,7))
sns.barplot(x="Dealer_No ",y="Avg_price_per_dealer", data = df_dealer,palette
plt.grid(axis = 'y',linestyle = '--',alpha=0.7)
plt.xticks(rotation =90)
plt.show()
```



```
In [155... #2 Which car brand (Company) has the highest variation in prices, and what doe
```

```
In [158... df.groupby('Company')['Price ($)'].var().sort_values(ascending = False).head(5
```

```
Out[158... Company
Lincoln      3.864389e+08
Saab         3.862695e+08
Cadillac     3.809180e+08
Plymouth     3.635123e+08
Lexus        3.187269e+08
Name: Price ($), dtype: float64
```

```
In [160... # Highest variance brand → “This company has the widest range of prices, which
#from low-end to high-end.”
```

```
In [162... #3 What is the distribution of car prices for each transmission type, and how
df.groupby("Transmission")['Price ($)'].describe()
```

	count	mean	std	min	25%	50%	
Transmission							
Auto	12571.0	28248.525972	13747.070597	1200.0	19000.0	24000.0	355
Manual	11335.0	27914.710631	15862.871978	1700.0	17000.0	22001.0	340

```
In [164... df.groupby("Transmission")["Price ($)"].apply(lambda x : x.quantile(0.75)-x.qu
```

```
Out[164... Transmission
Auto      16500.0
Manual    17000.0
Name: Price ($), dtype: float64
```

```
In [166... # Interpretation
# Both transmissions have very similar variability in the middle 50% of prices
# Manual cars have a slightly wider spread, meaning their prices fluctuate a b
# Auto cars are marginally more consistent in pricing.
```

```
In [168... #4 What is the distribution of car prices across different regions?
df.groupby("Dealer_Region")["Price ($)"].describe()
```

	count	mean	std	min	25%	50%	
Dealer_Region							
Aurora	3130.0	28334.626837	15026.207252	9000.0	18001.0	23000.0	350
Austin	4135.0	28341.603628	14903.884549	9000.0	18001.0	23801.0	350
Greenville	3128.0	28180.819054	15101.538328	1200.0	18001.0	22500.0	345
Janesville	3821.0	27833.350955	14344.995638	4300.0	18001.0	23000.0	340
Middletown	3128.0	27856.338875	14619.842395	1700.0	18000.0	22750.0	340
Pasco	3131.0	28119.039923	14659.315941	9000.0	18500.5	23000.0	340
Scottsdale	3433.0	27954.958928	14902.916820	1450.0	18000.0	22600.0	335

```
In [170... df.groupby("Dealer_Region")["Price ($)"].apply(lambda x: x.quantile(0.75)-x.qu
```

```
Out[170... Dealer_Region
Aurora      16999.0
Austin      17000.0
Greenville  16499.0
Janesville  15999.0
Middletown  16000.0
Pasco       15499.5
Scottsdale  15500.0
Name: Price ($), dtype: float64
```

```
In [172... #5 What is the distribution of cars based on body styles?
df['Body Style'].value_counts()
```

```
Out[172...] Body Style
SUV          6374
Hatchback    6128
Sedan        4488
Passenger    3945
Hardtop      2971
Name: count, dtype: int64
```

```
In [174...] #6 How does the average selling price of cars vary by customer gender and annu
df.groupby('Gender')[['Annual Income', 'Price ($)']].mean()
```

```
Out[174...]      Annual Income      Price ($)
Gender
Female  755973.021339  28277.265270
Male    851184.044207  28039.429407
```

```
In [176...] #7 What is the distribution of car prices by region, and how does the number c
df.groupby("Dealer_Region")["Price ($)"].describe()
```

```
Out[176...]      count      mean      std      min      25%      50%
Dealer_Region
Aurora    3130.0  28334.626837  15026.207252  9000.0  18001.0  23000.0  350
Austin    4135.0  28341.603628  14903.884549  9000.0  18001.0  23801.0  350
Greenville 3128.0  28180.819054  15101.538328  1200.0  18001.0  22500.0  345
Janesville 3821.0  27833.350955  14344.995638  4300.0  18001.0  23000.0  340
Middletown 3128.0  27856.338875  14619.842395  1700.0  18000.0  22750.0  340
Pasco     3131.0  28119.039923  14659.315941  9000.0  18500.5  23000.0  340
Scottsdale 3433.0  27954.958928  14902.916820  1450.0  18000.0  22600.0  335
```

```
In [178...] df["Dealer_Region"].value_counts()
```

```
Out[178...] Dealer_Region
Austin          4135
Janesville      3821
Scottsdale      3433
Pasco           3131
Aurora          3130
Middletown      3128
Greenville      3128
Name: count, dtype: int64
```

```
In [180...] #8 How does the average car price differ between cars with different engine si
df.groupby("Engine")["Price ($)"].mean()
```

```
Out[180...] Engine
DoubleÃ Overhead Camshaft    28248.525972
Overhead Camshaft           27914.710631
Name: Price ($), dtype: float64
```

```
In [182...] #9 How do car prices vary based on the customer's annual income bracket?
bins = [0, 50000, 200000, 500000, 1000000, df['Annual Income'].max()]
labels = ['Low', 'Lower-Mid', 'Mid', 'Upper-Mid', 'High']

df['Income_Bracket'] = pd.cut(df['Annual Income'], bins=bins, labels=labels, i
df.groupby('Income_Bracket')['Price ($)'].mean().round(2)
```

```
Out[182...] Income_Bracket
Low          27884.30
Lower-Mid    24094.06
Mid          28602.54
Upper-Mid    28011.73
High         28186.20
Name: Price ($), dtype: float64
```

```
In [202...] #10 What are the top 5 car models with the highest number of sales, and how do
top5_models = df['Model'].value_counts().head(5)
print(top5_models)
```

```
Model
Diamante      418
Silhouette    411
Prizm         411
Passat        391
Ram Pickup    383
Name: count, dtype: int64
```

```
In [206...] df[df['Model'].isin(top5_models.index)].groupby('Model')['Price ($)'].describe
```

```
Out[206...]      count      mean      std      min      25%      50%      75%
Model
Diamante  418.0  22268.303828  9770.768958  12000.0  18000.0  19001.0  22000.0
Passat    391.0  19600.728900  8218.521683   9000.0  12001.0  20000.0  27001.0
Prizm     411.0  22838.457421  8012.531045  15000.0  16000.0  22000.0  25000.0
Ram Pickup 383.0  27820.864230  13297.609534  16000.0  19000.0  21000.0  39300.0
Silhouette 411.0  29858.513382  17940.571762  12000.0  14001.0  25000.0  34500.0
```

```
In [222...] #11 How does car price vary with engine size across different car colors, and
df.groupby(['Color', 'Engine'])['Price ($)'].var()
```

```
Out[222... Color      Engine
Black      DoubleÃ Overhead Camshaft    1.542235e+08
           Overhead Camshaft          3.047610e+08
Pale White DoubleÃ Overhead Camshaft    2.074295e+08
           Overhead Camshaft          1.690027e+08
Red        DoubleÃ Overhead Camshaft    1.822326e+08
           Overhead Camshaft          3.009060e+08
Name: Price ($), dtype: float64
```

```
In [ ]: # Price variation by engine across colors:
# Overhead Camshaft engines generally have higher price variance than Double C
# Colors with highest variation:
# Black and Red cars with Overhead Camshaft engines show the most price fluctu
# Insight:
# If a customer is buying a Black or Red car with Overhead Camshaft, prices va
# Pale White cars are slightly more consistent in pricing.
```

```
In [226... #12 Is there any seasonal trend in car sales based on the date of sale?
df.Date.dtype
```

```
Out[226... dtype('O')
```

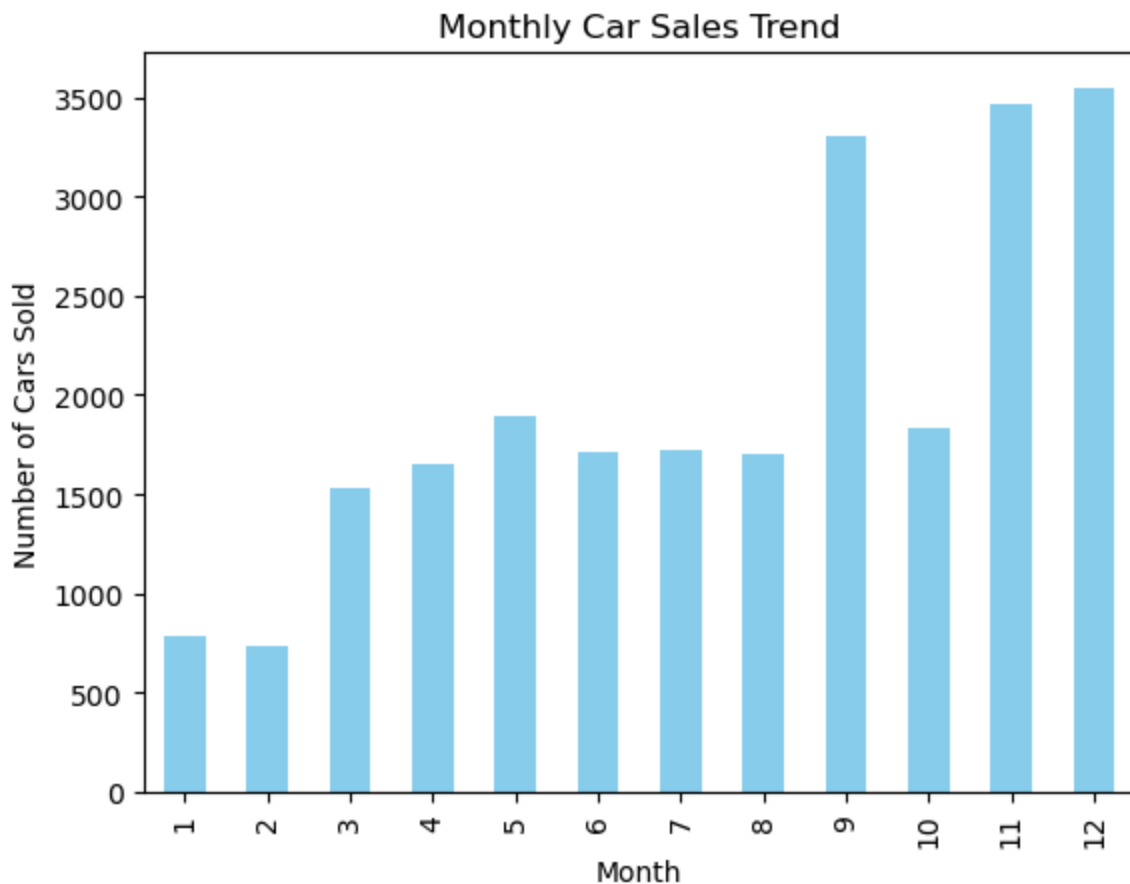
```
In [234... df["Date"] = pd.to_datetime(df["Date"])
```

```
In [242... df["Month"] = df["Date"].dt.month
```

```
In [247... monthly_sales = df['Month'].value_counts().sort_index()
print(monthly_sales)
```

```
Month
1      790
2      735
3     1535
4     1655
5     1895
6     1715
7     1725
8     1705
9     3305
10     1830
11     3470
12     3546
Name: count, dtype: int64
```

```
In [249... monthly_sales.plot(kind='bar', color='skyblue')
plt.xlabel('Month')
plt.ylabel('Number of Cars Sold')
plt.title('Monthly Car Sales Trend')
plt.show()
```



```
In [255...] # Insight = Maximum sale is in December
```

```
In [259...] #13 How does the car price distribution change when considering different combinations of body style and transmission?  
df.groupby(['Body Style', 'Transmission'])['Price ($)'].mean()
```

```
Out[259...] Body Style  Transmission  
Hardtop      Auto        31520.188210  
              Manual      27016.943698  
Hatchback    Auto        25910.544824  
              Manual      28702.550562  
Passenger    Auto        28915.835149  
              Manual      28969.521039  
SUV          Auto        27501.404407  
              Manual      26079.019161  
Sedan        Auto        29955.294344  
              Manual      29664.271572  
Name: Price ($), dtype: float64
```

```
In [282...] #14 What is the correlation between car price, engine size, and annual income?  
col = df[['Price ($)', 'Annual Income']]  
col.corr()  
  
#there is no column is given in dataset with engine size
```


Out[282...

	Price (\$)	Annual Income
Price (\$)	1.000000	0.012065
Annual Income	0.012065	1.000000

```
In [289... #15 How does the average car price vary across different car models and engine
avg_prices = df.groupby(['Model','Engine'])['Price ($)'].mean().unstack()
print(avg_prices)
```

Engine	DoubleÃ Overhead Camshaft	Overhead Camshaft
Model		
3-Sep	37986.380117	NaN
3000GT	22764.326923	21770.659864
300M	21394.888889	NaN
323i	16744.632287	21038.162162
328i	21069.149606	38676.177215
...	...	...
Viper	32118.479167	26052.375000
Voyager	22066.026316	33995.678322
Windstar	25145.636364	17100.272727
Wrangler	21145.294737	18742.942029
Xterra	15940.459459	31072.500000

[154 rows x 2 columns]

In []:

Title : EDA Over Amazon Sales Data



In []:

```
In [255... import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

In [257... df = pd.read_csv("amazon.csv")
df
```

Out[257...

	product_id	product_name	category
0	B07JW9H4J1	Wayona Nylon Braided USB to Lightning Fast Cha...	Computers&Accessories Accessories&Peripherals
1	B098NS6PVG	Ambrane Unbreakable 60W / 3A Fast Charging 1.5...	Computers&Accessories Accessories&Peripherals
2	B096MSW6CT	Source Fast Phone Charging Cable & Data Sync U...	Computers&Accessories Accessories&Peripherals
3	B08HDJ86NZ	boAt Deuce USB 300 2 in 1 Type-C & Micro USB S...	Computers&Accessories Accessories&Peripherals
4	B08CF3B7N1	Portronics Konnect L 1.2M Fast Charging 3A 8 P...	Computers&Accessories Accessories&Peripherals
...	...	...	
1460	B08L7J3T31	Noir Aqua - 5pcs PP Spun Filter + 1 Spanner ...	Home&Kitchen Kitchen&HomeAppliances WaterPuri
1461	B01M6453MB	Prestige Delight PRWO Electric Rice Cooker (1 ...	Home&Kitchen Kitchen&HomeAppliances SmallKitcl
1462	B009P2LIL4	Bajaj Majesty RX10 2000 Watts Heat Convector R...	Home&Kitchen Heating,Cooling&AirQuality RoomH
1463	B00J5DYCCA	Havells Ventil Air DSP 230mm Exhaust Fan (Pist...	Home&Kitchen Heating,Cooling&AirQuality Fans I
1464	B01486F4G6	Borosil Jumbo 1000-Watt Grill Sandwich Maker (...)	Home&Kitchen Kitchen&HomeAppliances SmallKitcl

1465 rows × 16 columns

In [258... `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1465 entries, 0 to 1464
Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype
---  -
0   product_id            1465 non-null   object
1   product_name          1465 non-null   object
2   category              1465 non-null   object
3   discounted_price      1465 non-null   object
4   actual_price          1465 non-null   object
5   discount_percentage   1465 non-null   object
6   rating               1465 non-null   object
7   rating_count          1463 non-null   object
8   about_product         1465 non-null   object
9   user_id              1465 non-null   object
10  user_name             1465 non-null   object
11  review_id            1465 non-null   object
12  review_title         1465 non-null   object
13  review_content       1465 non-null   object
14  img_link             1465 non-null   object
15  product_link         1465 non-null   object
dtypes: object(16)
memory usage: 183.3+ KB
```

In [259... *#1. What is the average rating for each product category?*

```
df.rating.value_counts()
#since data type is int we can convert it into int
df.rating = pd.to_numeric(df.rating,errors = 'coerce')
df.rating = df.rating.astype(float)
df.groupby("category")['rating'].mean().round(2)
```

```

Out[259... category
Car&Motorbike|CarAccessories|InteriorAccessories|AirPurifiers&Ionizers
3.80
Computers&Accessories|Accessories&Peripherals|Adapters|USBtoUSBAdapters
4.15
Computers&Accessories|Accessories&Peripherals|Audio&VideoAccessories|PCHeadse
ts 3.50
Computers&Accessories|Accessories&Peripherals|Audio&VideoAccessories|PCMicrop
hones 3.60
Computers&Accessories|Accessories&Peripherals|Audio&VideoAccessories|PCSpeake
rs 4.05

...
OfficeProducts|OfficePaperProducts|Paper|Stationery|Pens,Pencils&WritingSuppl
ies|Pens&Refills|GelInkRollerballPens 4.25
OfficeProducts|OfficePaperProducts|Paper|Stationery|Pens,Pencils&WritingSuppl
ies|Pens&Refills|LiquidInkRollerballPens 4.15
OfficeProducts|OfficePaperProducts|Paper|Stationery|Pens,Pencils&WritingSuppl
ies|Pens&Refills|RetractableBallpointPens 4.30
OfficeProducts|OfficePaperProducts|Paper|Stationery|Pens,Pencils&WritingSuppl
ies|Pens&Refills|StickBallpointPens 4.13
Toys&Games|Arts&Crafts|Drawing&PaintingSupplies|ColouringPens&Markers
4.30
Name: rating, Length: 211, dtype: float64

```

```

In [263... #2. What are the top rating_count products by category?
df['rating_count'] = df['rating_count'].str.replace(",", "").astype(float)

```

```

In [265... # 1. Find the index of top product in each category
idx = df.groupby('category')['rating_count'].idxmax()

# 2. Fetch the full rows using loc
top_products = df.loc[idx, ['category', 'product_name', 'rating_count']]

# 3. Optional: sort overall by rating_count descending
top_products = top_products.sort_values(by='rating_count', ascending=False).re
top_products

```

Out[265...

	category	product_name	rating_coun
0	Electronics HomeTheater,TV&Video Accessories C...	AmazonBasics Flexible Premium HDMI Cable (Blac...	426973.
1	Electronics Headphones,Earbuds&Accessories Hea...	boAt Bassheads 100 in Ear Wired Earphones with...	363713.
2	Electronics Mobiles&Accessories Smartphones&Ba...	Redmi 9 Activ (Carbon Black, 4GB RAM, 64GB Sto...	313836.
3	Home&Kitchen Kitchen&Dining KitchenTools Manua...	Pigeon Polypropylene Mini Handy and Compact Ch...	270563.
4	Computers&Accessories ExternalDevices&DataStor...	SanDisk Cruzer Blade 32GB USB Flash Drive	253105.
...	...	...	...
206	Computers&Accessories Components InternalHardD...	Lapster Caddy for ssd and HDD, Optical Bay 2nd...	362.
207	Computers&Accessories Laptops TraditionalLaptops	Lenovo IdeaPad 3 11th Gen Intel Core i3 15.6" ...	323.
208	Electronics Mobiles&Accessories MobileAccessor...	LAPSTER 12pcs Spiral Cable Protectors for Char...	305.
209	Electronics HomeAudio Speakers TowerSpeakers	KRISONS Thunder Speaker, Multimedia Home Theat...	282.
210	Electronics HomeAudio MediaStreamingDevices St...	Realme Smart TV Stick 4K	224.

211 rows × 3 columns

In [267...

```
#3. What is the distribution of discounted prices vs. actual prices?
df['discounted_price'] = df['discounted_price'].str.replace(r"[\₹,]", "", regex=True)
df['actual_price'] = df['actual_price'].str.replace(r"[\₹,]", "", regex=True).a
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1465 entries, 0 to 1464
Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype
---  -
0   product_id            1465 non-null   object
1   product_name          1465 non-null   object
2   category              1465 non-null   object
3   discounted_price      1465 non-null   float64
4   actual_price          1465 non-null   float64
5   discount_percentage   1465 non-null   object
6   rating                1464 non-null   float64
7   rating_count          1463 non-null   float64
8   about_product         1465 non-null   object
9   user_id               1465 non-null   object
10  user_name             1465 non-null   object
11  review_id             1465 non-null   object
12  review_title          1465 non-null   object
13  review_content        1465 non-null   object
14  img_link              1465 non-null   object
15  product_link          1465 non-null   object
dtypes: float64(4), object(12)
memory usage: 183.3+ KB

```

```
In [269...] df.discounted_price
```

```

Out[269...] 0      399.0
            1      199.0
            2      199.0
            3      329.0
            4      154.0
            ...
1460      379.0
1461     2280.0
1462     2219.0
1463     1399.0
1464     2863.0
Name: discounted_price, Length: 1465, dtype: float64

```

```
In [271...] df.actual_price
```

```

Out[271...] 0      1099.0
            1       349.0
            2     1899.0
            3       699.0
            4       399.0
            ...
1460       919.0
1461     3045.0
1462     3080.0
1463     1890.0
1464     3690.0
Name: actual_price, Length: 1465, dtype: float64

```

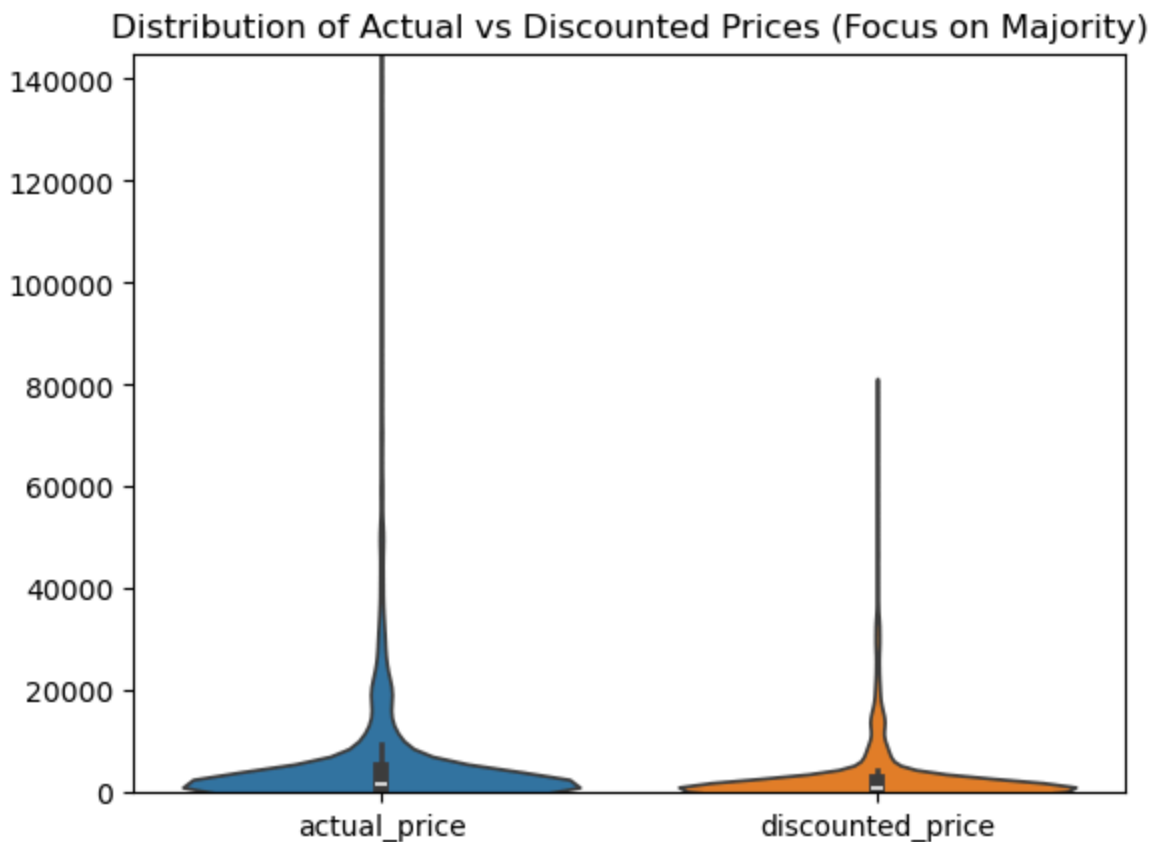
```
In [273... df.discounted_price.describe()
```

```
Out[273... count      1465.000000  
mean       3125.310874  
std        6944.304394  
min         39.000000  
25%        325.000000  
50%        799.000000  
75%       1999.000000  
max       77990.000000  
Name: discounted_price, dtype: float64
```

```
In [275... df.actual_price.describe()
```

```
Out[275... count      1465.000000  
mean       5444.990635  
std       10874.826864  
min         39.000000  
25%        800.000000  
50%       1650.000000  
75%       4295.000000  
max      139900.000000  
Name: actual_price, dtype: float64
```

```
In [277... sns.violinplot(data=df[['actual_price', 'discounted_price']])  
# plt.xticks([0,1], ['Actual Price', 'Discounted Price'])  
plt.ylim(0, 145000) # adjust according to your typical price range  
plt.title("Distribution of Actual vs Discounted Prices (Focus on Majority)")  
plt.show()
```

In [278... *#4. How does the average discount percentage vary across categories?*
`df.discount_percentage = df.discount_percentage.str.strip("%")`

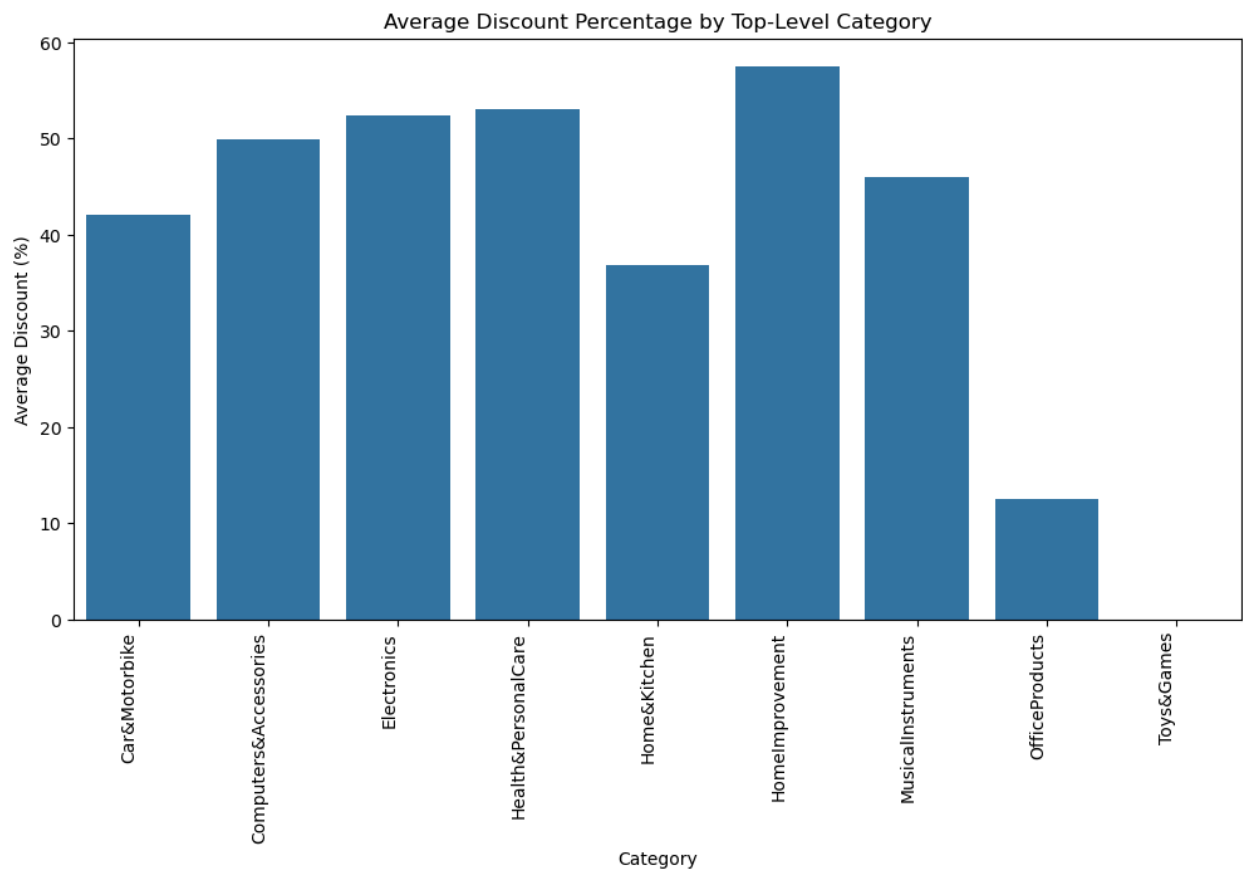
In [281... `df.discount_percentage = df.discount_percentage.astype(float)`

In [283... `df1 = df.groupby('category')['discount_percentage'].mean().reset_index(name =
df1 = df1.sort_values(by='avg_discount', ascending=False).reset_index(drop=True)
df1.columns`

Out[283... `Index(['category', 'avg_discount'], dtype='object')`

In [285... `df1['top_category'] = df1['category'].apply(lambda x: x.split('|')[0])
category_discount = df1.groupby('top_category')['avg_discount'].mean().reset_i
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(12,6))
sns.barplot(x='top_category', y='avg_discount', data=category_discount)
plt.xticks(rotation=90, ha='right')
plt.title('Average Discount Percentage by Top-Level Category')
plt.ylabel('Average Discount (%)')
plt.xlabel('Category')
plt.show()`



In [286... *#5 What are the most popular product names?*
`df[['product_name', 'rating_count']].sort_values(by = 'rating_count', ascending`

Out[286... **product_name** **rating_count**

0	AmazonBasics Flexible Premium HDMI Cable (Blac...	426973.0
1	Amazon Basics High-Speed HDMI Cable, 6 Feet (2...	426973.0
2	Amazon Basics High-Speed HDMI Cable, 6 Feet - ...	426973.0
3	AmazonBasics Flexible Premium HDMI Cable (Blac...	426972.0
4	boAt Bassheads 100 in Ear Wired Earphones with...	363713.0
5	boAt Bassheads 100 in Ear Wired Earphones with...	363713.0
6	boAt BassHeads 100 in-Ear Wired Headphones wit...	363711.0
7	Redmi 9 Activ (Carbon Black, 4GB RAM, 64GB Sto...	313836.0
8	Redmi 9A Sport (Coral Green, 2GB RAM, 32GB Sto...	313836.0
9	Redmi 9A Sport (Carbon Black, 2GB RAM, 32GB St...	313832.0

In [289... *#6. What are the most popular product keywords?*
`df['product_name'].head()
df['tokens'] = df['product_name'].str.lower().str.split()
all_words = df['tokens'].explode()`

```
In [291...] stopwords = ["to", "and", "with", "for", "of", "in", "on", "by", "an", "a", "t"]
keywords = all_words[~all_words.isin(stopwords)]
```

```
In [293...] keywords.value_counts().head(10)
```

```
Out[293...] tokens
usb          377
cable        320
charging     219
fast         211
(black)      187
c            182
smart        171
type         144
compatible   138
tv           138
Name: count, dtype: int64
```

```
In [317...] #7. What are the most popular product reviews?
top_products = df.sort_values(by="rating_count", ascending=False).head(5)

popular_reviews = top_products[["product_name", "rating_count", "review_title"]
popular_reviews.reset_index(drop = True)
```

```
Out[317...]
      product_name  rating_count  review_title  review_content
0  AmazonBasics Flexible Premium HDMI Cable (Blac...  426973.0  It's quite good and value for money,Works well...  I am using it for 14 days now. The experience ...
1  Amazon Basics High-Speed HDMI Cable, 6 Feet (2...  426973.0  It's quite good and value for money,Works well...  I am using it for 14 days now. The experience ...
2  Amazon Basics High-Speed HDMI Cable, 6 Feet - ...  426973.0  It's quite good and value for money,Works well...  I am using it for 14 days now. The experience ...
3  AmazonBasics Flexible Premium HDMI Cable (Blac...  426972.0  It's quite good and value for money,Works well...  I am using it for 14 days now. The experience ...
4  boAt Bassheads 100 in Ear Wired Earphones with...  363713.0  Best value for money,HEAD PHONE POUCH NOT RECE...  The sound quality of this earphone are really ...
```

```
In [322...] #8. What is the correlation between discounted_price and rating?
df[['discounted_price', 'rating']].corr()
```

```
Out[322...]
      discounted_price  rating
discounted_price      1.000000  0.120337
rating                0.120337  1.000000
```

```
In [332... #9. What are the Top 5 categories based on the highest ratings?  
df.groupby("category")['rating'].max().sort_values(ascending = False).head(5)
```

```
Out[332... category  
Computers&Accessories|Accessories&Peripherals|Keyboards,Mice&InputDevices|Mic  
e          5.0  
Computers&Accessories|Accessories&Peripherals|Cables&Accessories|Cables|USBCa  
bles       5.0  
Home&Kitchen|Kitchen&HomeAppliances|SmallKitchenAppliances|HandBlenders  
4.8  
Home&Kitchen|Heating,Cooling&AirQuality|WaterHeaters&Geysers|InstantWaterHeat  
ers        4.8  
Home&Kitchen|Kitchen&HomeAppliances|SmallKitchenAppliances|DeepFatFryers|AirF  
ryers      4.8  
Name: rating, dtype: float64
```

```
In [334... # 10. Identify any potential areas for improvement or optimization based on th  
# ANS.  
# Pricing Optimization – The correlation between discounted price and ratings  
# customer satisfaction. Discounts should be targeted towards low-performing p  
  
# Product Quality in Certain Categories – Categories with consistently low rat  
# These should be addressed either by improving product quality, working with  
  
# Customer Feedback Utilization – Reviews highlight common concerns such as pr  
# Addressing these recurring issues can directly improve customer satisfaction.  
  
# Review Engagement – Some products show high ratings but very few reviews. En  
# improve purchase confidence for new customers.  
  
# Category-Level Focus – Categories with higher ratings should receive more ma  
# high sales but lower ratings should be prioritized for quality improvement.
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

Title: EDA Over Spotify Dataset



In []:

```
In [6]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
```

```
In [10]: df = pd.read_csv("spotify.csv")
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 440 entries, 0 to 439
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Artist          440 non-null   object
1   Track Name      440 non-null   object
2   Popularity      440 non-null   int64
3   Duration (ms)  440 non-null   int64
4   Track ID        440 non-null   object
dtypes: int64(2), object(3)
memory usage: 17.3+ KB
```

```
In [12]: df
```

Out[12]:

	Artist	Track Name	Popularity	Duration (ms)	Track ID
0	Drake	Rich Baby Daddy (feat. Sexyy Red & SZA)	92	319191	1yeB8MUNeLo9Ek1UEpsyz6
1	Drake	One Dance	91	173986	1zi7xx7UVEFkmKfv06H8x0
2	Drake	IDGAF (feat. Yeat)	90	260111	2YSzYUF3jWqb9YP9VXmpjE
3	Drake	First Person Shooter (feat. J. Cole)	88	247444	7aqfrAY2p9BUSiupwk3svU
4	Drake	Jimmy Cooks (feat. 21 Savage)	88	218364	3F5CgOj3wFIRv51JsHbxhe
...	...	...	...	...	...
435	French Montana	Splash Brothers	44	221863	3fBsEOnzwtlkpS0LxXAZhN
436	Fat Joe	All The Way Up (feat. Infared)	64	191900	7Ezwtgfw7khBrpvNPtMoT
437	A\$AP Ferg	Work REMIX (feat. A\$AP Rocky, French Montana, ...)	69	283693	7xVLFuuYdAvctfcP3IG3dS
438	Diddy	Another One Of Me (feat. 21 Savage)	65	220408	4hGmQboiou09EwhcTWa0H6
439	Rick Ross	Stay Schemin	68	267720	0nq6sfr8z1R5KJ4XUk396e

440 rows × 5 columns

```
In [14]: # 1 Read the dataframe, check null value if present then do the needful, check
# the needful?
df.isnull().sum()
```

```
Out[14]: Artist      0
Track Name    0
Popularity    0
Duration (ms)  0
Track ID      0
dtype: int64
```

```
In [16]: #there is no null null values present in dataset
```

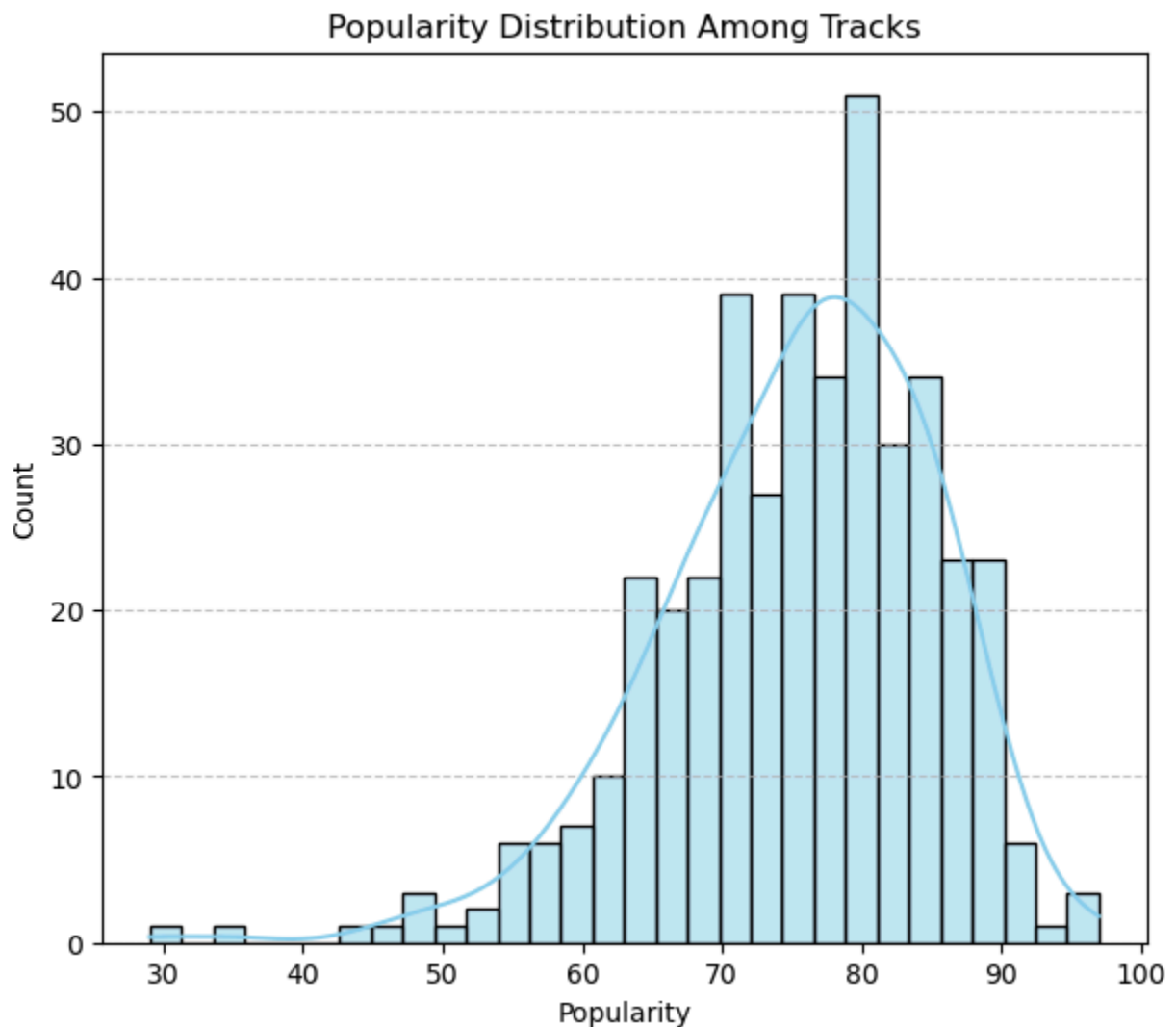
```
In [35]: df.drop_duplicates(inplace = True)
```

```
In [37]: df.duplicated().sum()
```

```
Out[37]: 0
```

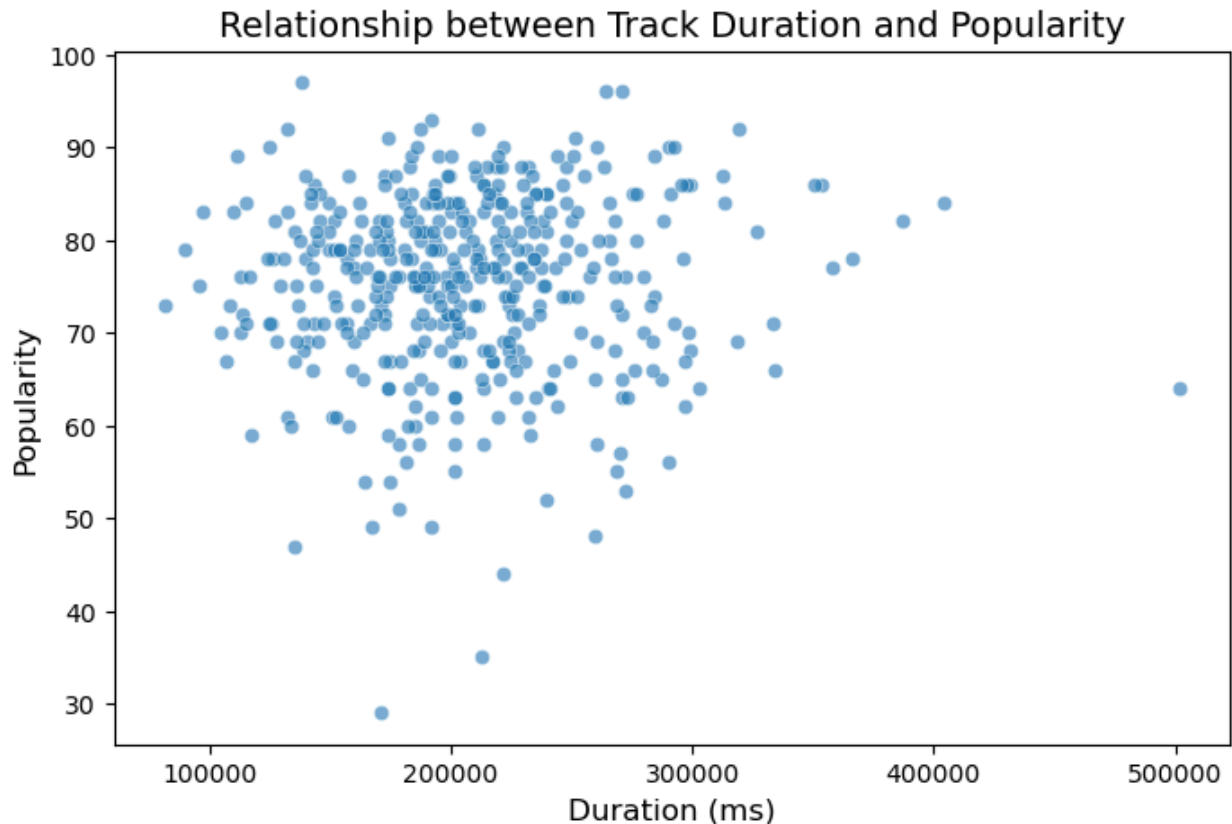
```
In [39]: #Duplicate values were identified and removed, ensuring the dataset now contains
```

```
In [99]: # 2 What is the distribution of popularity among the tracks in the dataset? Visualize the distribution using a histogram and KDE plot.  
plt.figure(figsize = (7,6))  
sns.histplot(df['Popularity'],bins=30,kde = True,color = 'skyblue')  
plt.title("Popularity Distribution Among Tracks")  
plt.grid(axis = 'y',linestyle = '--',alpha=0.7)  
plt.show()
```



```
In [119]: # 3 Is there any relationship between the popularity and the duration of track? Visualize the relationship using a scatter plot.  
plt.figure(figsize=(8,5))  
sns.scatterplot(x='Duration (ms)', y='Popularity', data=df, alpha=0.6)  
  
plt.title("Relationship between Track Duration and Popularity", fontsize=14)  
plt.xlabel("Duration (ms)", fontsize=12)  
plt.ylabel("Popularity", fontsize=12)
```

```
plt.show()
```



```
In [ ]: # The points are scattered all over without forming a strong upward or downward trend
# This suggests weak or no strong correlation between track duration and popularity
# The scatter plot indicates no strong relationship between track duration and popularity
# between 2.5 to 4 minutes, while extremely long tracks are relatively less popular
# a weak association.
```

```
In [127]: df[['Duration (ms)', 'Popularity']].corr()
```

```
Out[127]:
```

	Duration (ms)	Popularity
Duration (ms)	1.000000	0.038992
Popularity	0.038992	1.000000

```
In [129]: # The correlation coefficient between track duration and popularity is 0.04, indicating a weak positive correlation.
# This confirms that track length does not significantly influence popularity.
```

```
In [145]: # 4 Which artist has the highest number of tracks in the dataset? Display the top 5 artists.
df['Artist'].value_counts()
```



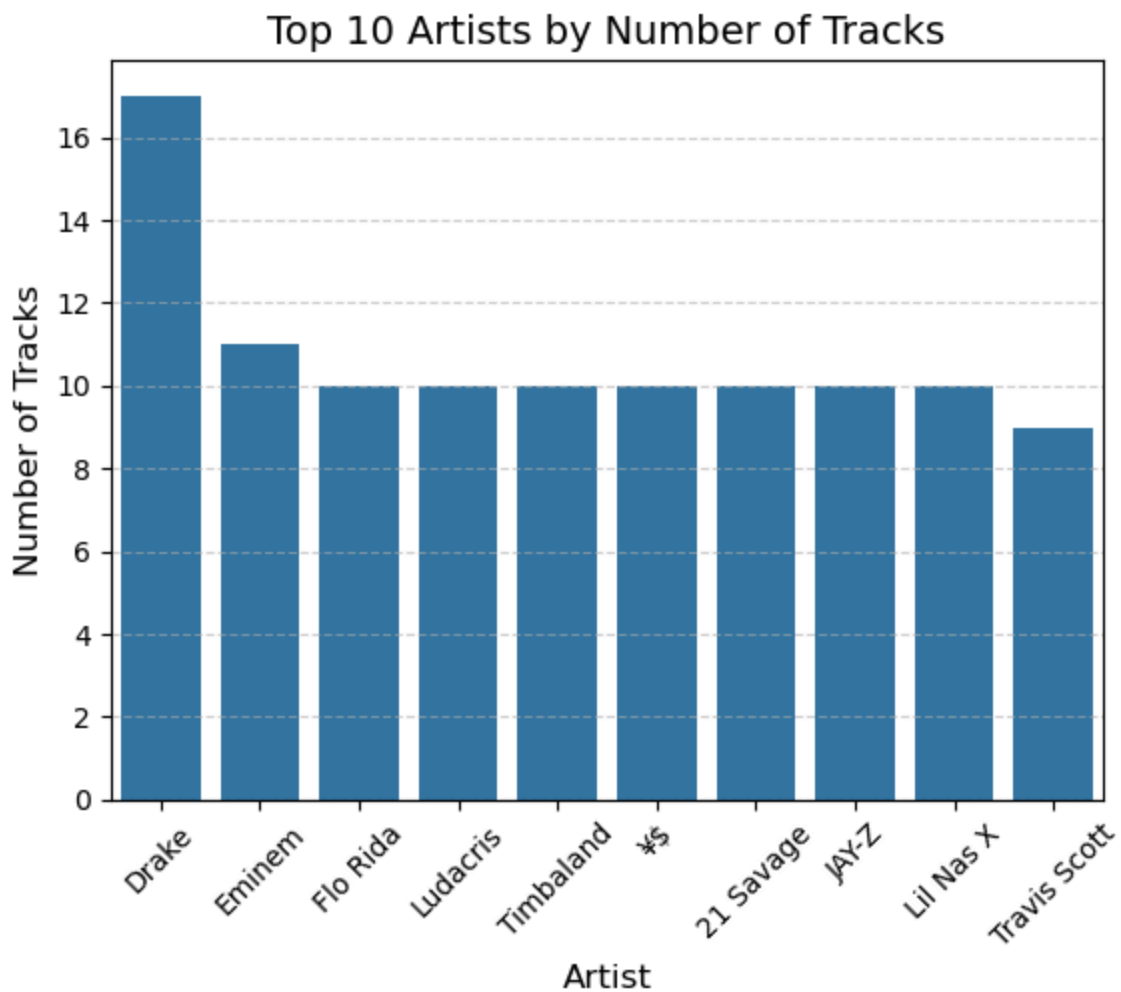
```
Out[145... Artist
Drake          17
Eminem         11
Flo Rida       10
Ludacris       10
Timbaland      10
..
Arizona Zervas 1
Fivio Foreign  1
Pressa         1
David Guetta   1
Diddy          1
Name: count, Length: 115, dtype: int64
```

```
In [ ]: #Drake is the artist who has highest number of tracks in the dataset with 17 t
```

```
In [151... #since number of artist are 115 , its not good visualisation using countplot w
# with respect to their Tracks
top_artist = df['Artist'].value_counts().head(10)
top_artist
```

```
Out[151... Artist
Drake          17
Eminem         11
Flo Rida       10
Ludacris       10
Timbaland      10
¥$            10
21 Savage     10
JAY-Z         10
Lil Nas X     10
Travis Scott   9
Name: count, dtype: int64
```

```
In [163... sns.countplot(x='Artist', data=df,order=df['Artist'].value_counts().index[:10]
plt.title("Top 10 Artists by Number of Tracks", fontsize=14)
plt.xlabel("Artist", fontsize=12)
plt.ylabel("Number of Tracks", fontsize=12)
plt.xticks(rotation=45)
plt.grid(axis = 'y',linestyle='--',alpha =0.6)
plt.show()
```



In [173... *# 5 What are the top 5 least popular tracks in the dataset? Provide the artist*
`df[['Artist','Popularity','Track Name']].sort_values(by = 'Popularity',ascendi`

Out[173...

	Artist	Popularity	Track Name
207	Pressa	29	Attachments (feat. Coi Leray)
231	Justin Bieber	35	Intentions
413	French Montana	44	Splash Brothers
225	Lil Baby	47	On Me - Remix
407	Wyclef Jean	48	911 (feat. Mary J. Blige)

In [181... *#6 Among the top 5 most popular artists, which artist has the highest populari*
#Calculate and display the average popularity for each artist?
`df.groupby('Artist')['Popularity'].mean().sort_values(ascending = False).head(`

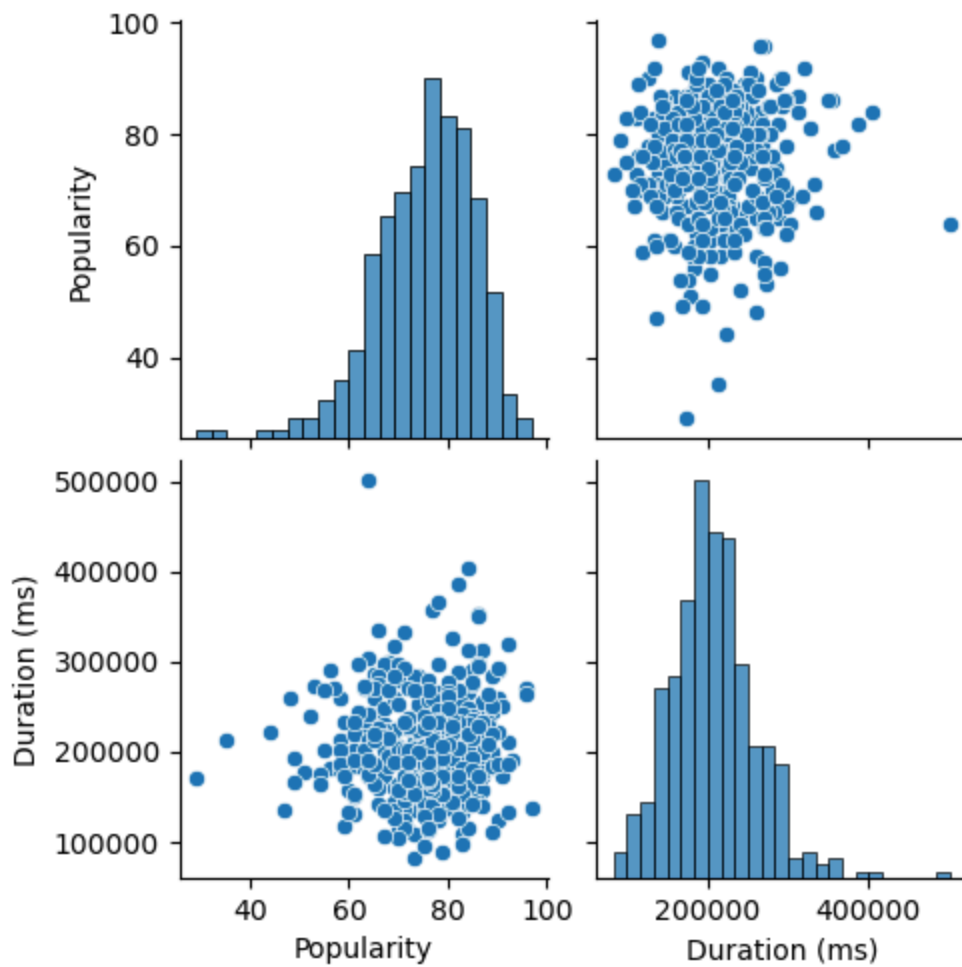
```
Out[181...] Artist
cassö          92.00
Trueno         89.00
David Guetta   87.00
Travis Scott   86.56
¥$            85.10
Name: Popularity, dtype: float64
```

```
In [183...] #So, the artist with the highest average popularity is cassö (92.00).
```

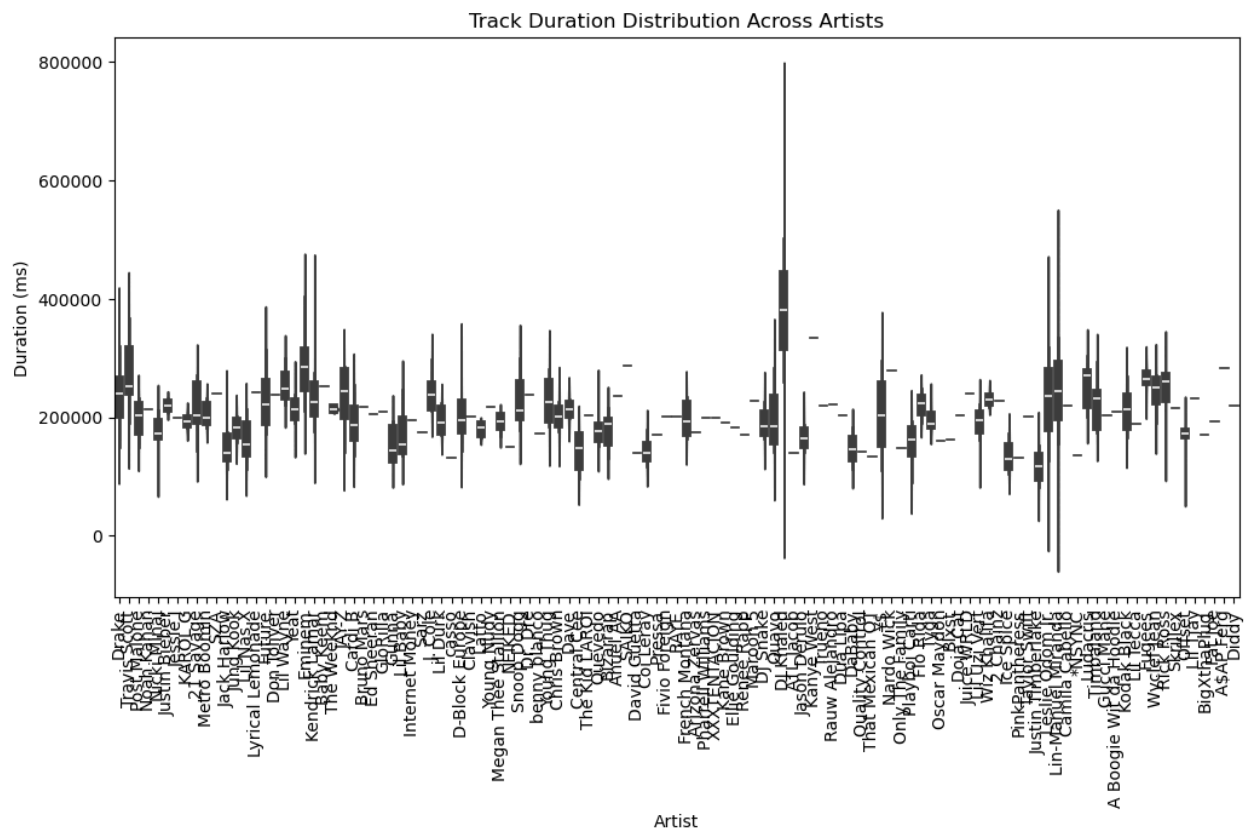
```
In [197...] # 7 For the top 5 most popular artists, what are their most popular tracks? List
df[['Artist', 'Popularity', 'Track Name']].sort_values(by='Popularity', ascending
```

```
Out[197...]
   Artist  Popularity  Track Name
0  Jack Harlow      97    Lovin On Me
1   21 Savage      96         redrum
2      ¥$          96       CARNIVAL
3  Travis Scott      93  FE!N (feat. Playboi Carti)
4     cassö        92          Prada
```

```
In [199...] # 8 Visualize relationships between multiple numerical variables simultaneously
numerical_cols = ['Popularity', 'Duration (ms)']
sns.pairplot(df[numerical_cols])
plt.show()
```

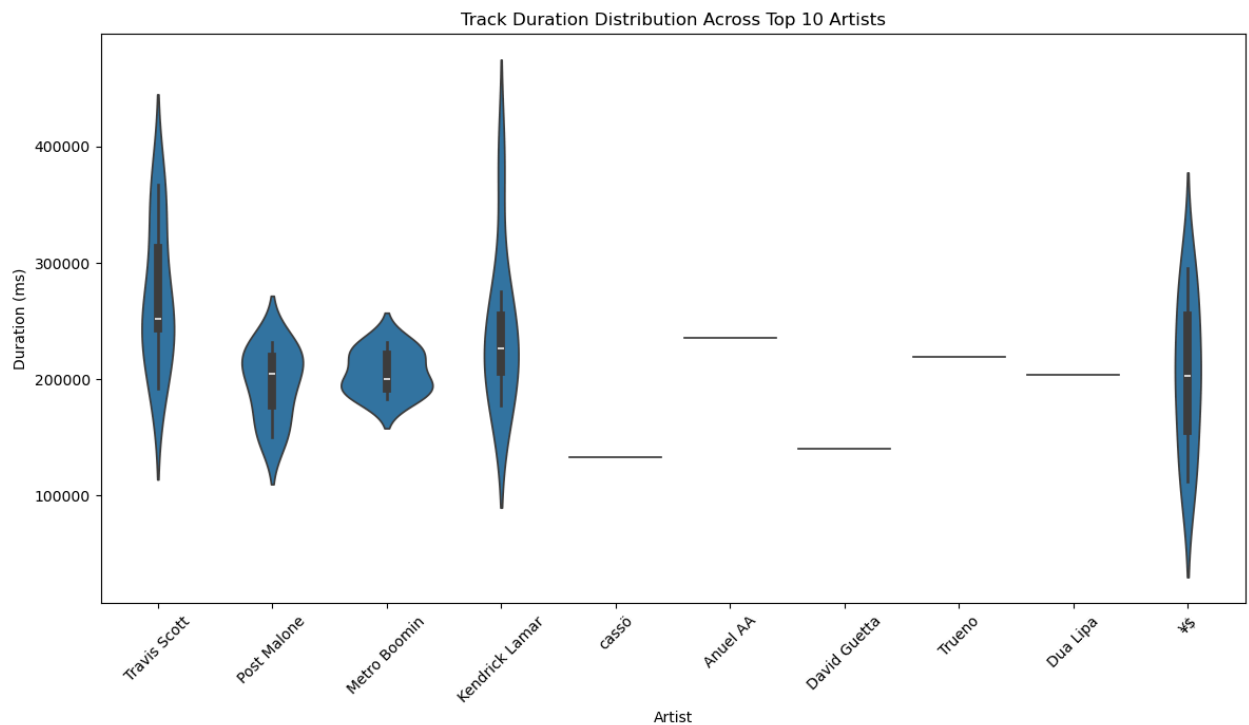


```
In [205... # 9 Does the duration of tracks vary significantly across different artists? E
plt.figure(figsize=(12,6))
sns.violinplot(x='Artist', y='Duration (ms)', data=df)
plt.xticks(rotation=90)
plt.title("Track Duration Distribution Across Artists")
plt.show()
```



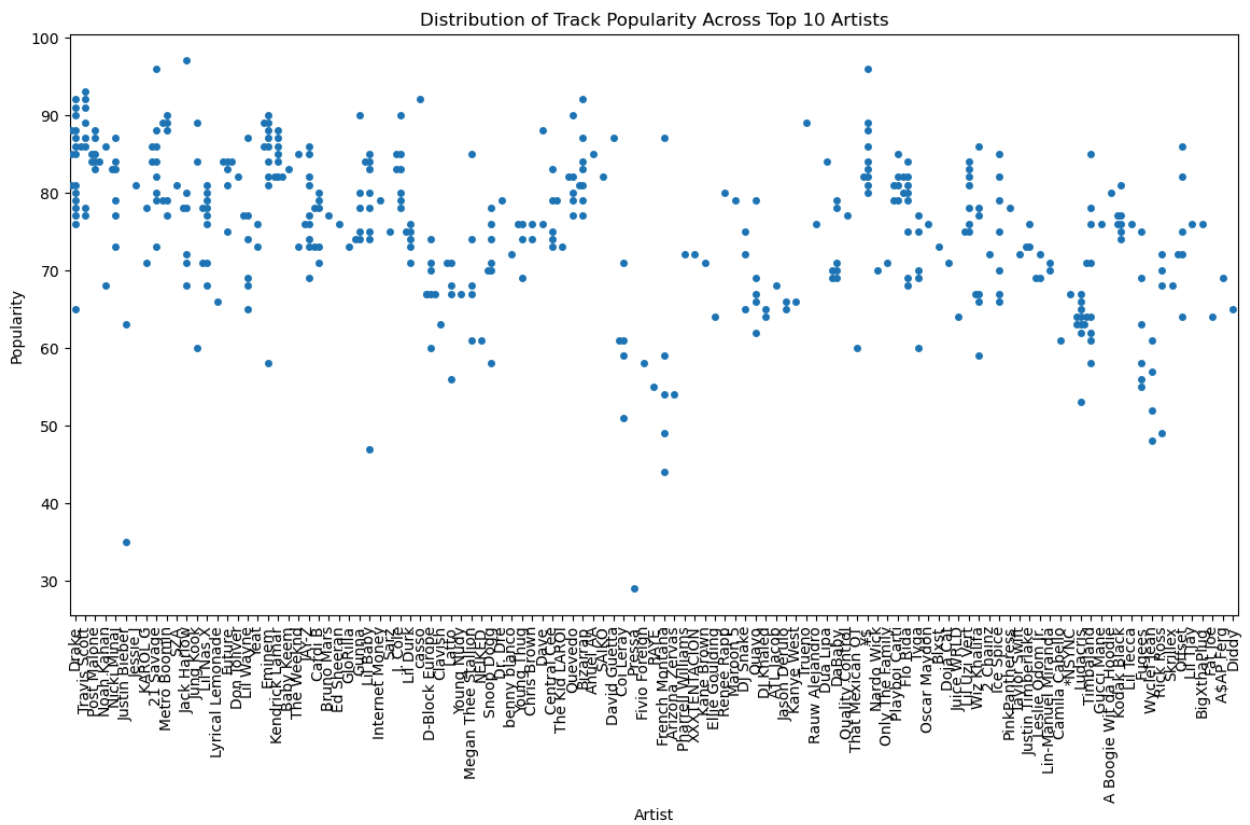
In [209... *#we have many artists, it's better to focus on the top 5 or top 10 most popular*

```
top_artists = df.groupby('Artist')['Popularity'].mean().sort_values(ascending=
top_artists_df = df[df['Artist'].isin(top_artists)]
plt.figure(figsize=(14,7))
sns.violinplot(x='Artist', y='Duration (ms)', data=top_artists_df)
plt.xticks(rotation=45) # Rotate labels for readability
plt.title("Track Duration Distribution Across Top 10 Artists")
plt.ylabel("Duration (ms)")
plt.xlabel("Artist")
plt.show()
```



In [218... *# 10 How does the distribution of track popularity vary for different artists?*
violin plot?

```
plt.figure(figsize=(14,7))
sns.swarmplot(x='Artist', y='Popularity', data=df, size=5)
plt.xticks(rotation=90)
plt.title("Distribution of Track Popularity Across Top 10 Artists")
plt.ylabel("Popularity")
plt.xlabel("Artist")
plt.show()
```



In []: